

PA1 — A* Search and Heuristics with DOGCAT

Name: Sean Raynor Section: 1

Overview

This document summarizes the implemented DC problem, heuristics, experiments, and answers to the written questions for PA1.

Implementation notes

- `dc.py`: implements DC extending `search.Problem`.
- Cost measures implemented: `steps`, `scrabble`, `frequency`.
 - `steps`: each action costs 1.
 - `scrabble`: cost equals Scrabble value of inserted letter.
 - `frequency`: cost equals $1 + \text{rarity}(w)$ where $\text{rarity}(w)$ is the nonnegative value from `words34.txt.gz`.
- Heuristics:
 - `h_steps(n) = HammingDistance(n, goal)` (admissible).
 - `h_scrabble(n) = sum(scrabble_score(goal[i]) for differing positions i)` (admissible).
 - `h_frequency(n) = mismatches * 1` (baseline admissible; used as `h0`).
 - `h1_frequency(n) = mismatches * (1 + min_rarity_len)` (rarity-aware lower bound tested experimentally).

Formal statements and admissibility proofs

1. Steps heuristic

- Formula: $h_steps(n) = |\{i : n[i] \neq goal[i]\}|$.
- Lower bound justification: Each legal action changes exactly one letter, so to correct k mismatches you need at least k actions. Each action costs 1 under the `steps` objective, so the remaining cost is at least k . Thus $h_steps \leq \text{true remaining cost}$. $h_steps(goal) = 0$.

2. Scrabble heuristic

- Formula: $h_scrabble(n) = \sum_{i : n[i] \neq goal[i]} \text{scrabble_score}(goal[i])$.
- Lower bound justification: To make position i match the goal, at some point an action must set that position to the goal letter (or pass through an equally or more expensive intermediate letter). The minimal possible cost

contribution to fix position i is at least the scrabble value of the cheapest letter that can occupy that position in an optimal sequence; using `scrabble_score(goal[i])` is conservative (it assumes the cheapest way to end up with the goal letter is to insert the goal letter directly). Therefore the sum over differing positions lower-bounds the remaining scrabble cost. $h_scrabble(goal)=0$ and it cannot overestimate the minimal remaining scrabble cost.

3. Frequency heuristic (baseline h_0)

- Formula: $h_0(n) = \text{mismatches} * 1$.
- Lower bound justification: Under the frequency objective every action that produces a new word w has cost $1 + \text{rarity}(w) \geq 1$ (because $\text{rarity}(w) \geq 0$). Therefore each required remaining action contributes at least 1 to the path cost, so k mismatches implies at least k cost remaining. $h_0(goal)=0$ and it never overestimates.

4. Rarity-aware heuristic (h_1)

- Candidate formula: $h_1(n) = \text{mismatches} * (1 + r_min_len)$, where $r_min_len = \min\{ \text{rarity}(w) : w \text{ in dictionary and } len(w) = L \}$ and L is word length.
- Lower bound justification: Every action that preserves length produces a 3- or 4-letter word; for words of length L , the rarity term of any produced word is at least r_min_len . Hence each action costs at least $1 + r_min_len$, so k mismatches imply remaining cost $\geq k * (1 + r_min_len)$. Therefore h_1 is admissible. It is nontrivial and strictly larger than h_0 for any instance where $r_min_len > 0$.

Scrabble scores used

A, E, I, O, U, L, N, S, T, R = 1 D, G = 2 B, C, M, P = 3 F, H, V, W, Y = 4 K = 5 J, X = 6 Q, Z = 10

Experiments

Ran UCS and A* (with implemented heuristics) for the pairs `WHY->ASK` and `BEAR->DUCK` under each cost measure.

Results (pair,cost,UCS_cost,UCS_expanded,A*_cost,A*_expanded,Reduction%):

```
why->ask, steps, 5, 886, 5, 63, 92.89
why->ask, scrabble, 9, 774, 9, 34, 95.61
why->ask, frequency (h0), 38.85540652, 808, 38.85540652, 755, 6.56
why->ask, frequency (h1), 38.85540652, 808, 38.85540652, 754, 6.68
bear->duck, steps, 4, 1226, 4, 6, 99.51
bear->duck, scrabble, 11, 3113, 11, 6, 99.81
bear->duck, frequency (h0), 38.2402284, 1084, 38.2402284, 720, 33.58
bear->duck, frequency (h1), 38.2402284, 1084, 38.2402284, 631, 41.79
```

Observations:

- A* reduced node expansions substantially for `steps` and `scrabble` (often >90%).
- For `frequency`, h_1 (rarity-aware) improved reductions vs baseline h_0 , particularly for `BEAR->DUCK`.
- All runs returned the same optimal costs for UCS and A*.

Answers to written questions (brief)

Q1 (heuristics):

- Steps: $h(n)$ = number of differing positions — lower-bounds remaining steps since each action changes at most one letter.
- Scrabble: $h(n)$ = $\sum(\text{scrabble_score}(\text{goal}[i]) \text{ for differing positions } i)$ — lower-bounds remaining scrabble cost because each differing position requires inserting at least the goal letter or an intermediate letter not cheaper than the goal letter at that position (conservative bound).
- Frequency: $h_0(n)$ = mismatches * 1 — each differing position requires at least one action costing at least 1; admissible. $h_1(n)$ = mismatches * $(1 + \text{min_rarity_len})$ uses the minimum rarity among words of the same length as a potential tighter lower bound.

Q2 (UCS vs A* experiments): see table above and analysis in Observations.

Q3 (reversibility):

- Steps: Yes — symmetric; path length same reversed.
- Scrabble: No — replacement letter costs depend on inserted letter; reversing may replace different letters and give different total cost (example: bear→duck vs duck→bear).
- Frequency: No — costs depend on rarities of intermediate words; asymmetric in general.

Q4 (LLM heuristic audit):

- Prompt used: "In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.". Proposed LLM heuristic was a rarity-aware multiplier of Hamming distance; I tested and repaired it to h_1 above and validated experimentally on the two frequency tasks.

Model used: GPT-5 mini

Files changed/added

- `dc.py` (implemented frequency cost and heuristics)
- `run_ucs_vs_astar.py` (experiment script)
- `pa01.md` (this draft)

How to reproduce

Run tests and experiments:

```
python dctest.py  
python run_ucs_vs_astar.py  
python dcsolve.py dog cat steps
```

